DevOps Central Platform

Architecture & Module Study

An inventory of the platform's functional modules and submodules, with architecture diagrams showing how the modules connect across the React single-page frontend and the Express/Prisma backend.

Repository	devops (branch: azfarz)
Backend	Node.js 20 · Express 4 · TypeScript · Prisma 5 · PostgreSQL · Redis · Socket.IO
Frontend	React 18 · Vite · TypeScript · Tailwind CSS · Radix/shadcn UI · Zustand · TanStack Query
Realtime	Socket.IO (shared HTTP server, ticket comments / task board / timesheet events)
Deployment	Docker Compose: frontend, backend, PostgreSQL 15, Redis 7, Nginx reverse proxy
Scale	33 backend route modules · ~74 Prisma models · 8 functional modules

1 System Overview

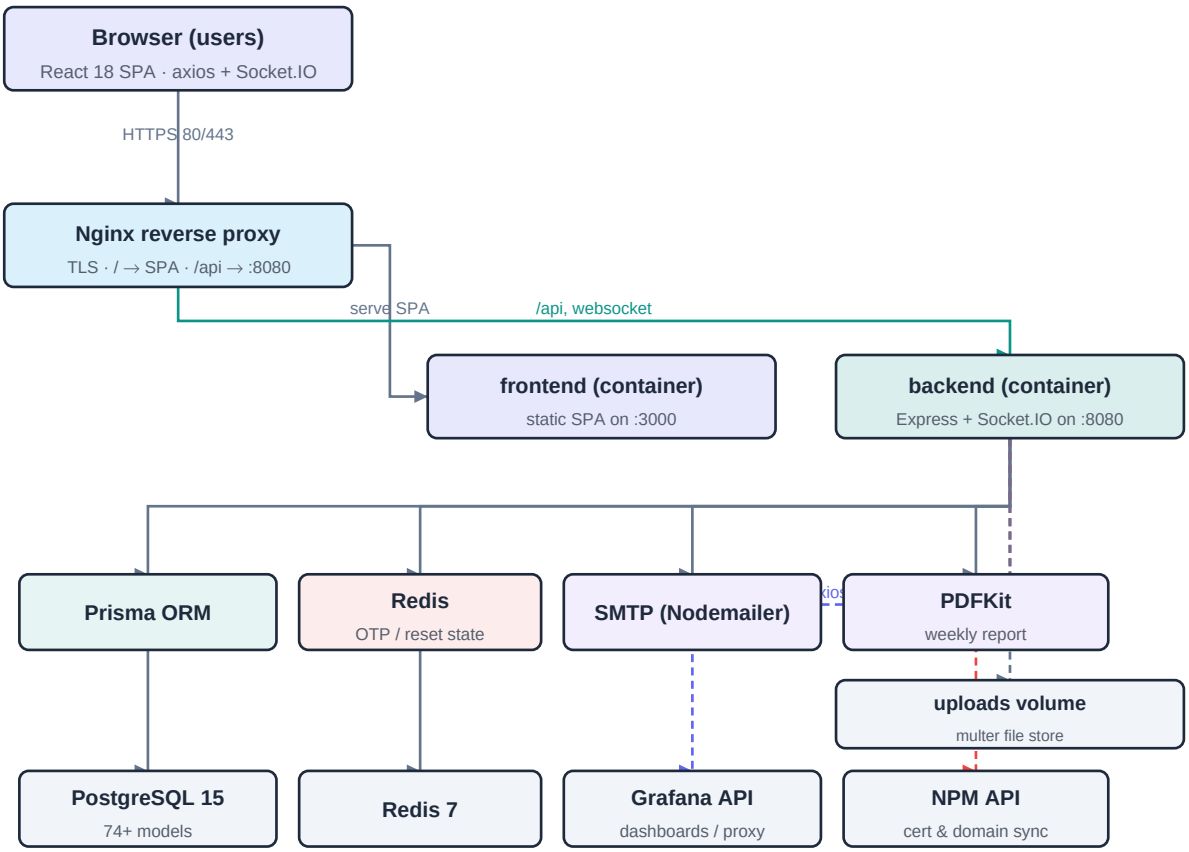
The platform is a centralized internal DevOps operations and ERP-style suite. A React SPA talks to a single Express API over REST (axios, JWT in httpOnly cookies) plus Socket.IO for realtime events. The backend is a modular monolith: 33 route modules grouped by domain, all talking directly to PostgreSQL through Prisma, with cross-module behavior delegated to a small set of shared services (notifications, email, approval engine, activity logging, reference numbering).

1.1 Technology stack

Layer	Technology	Notes
Frontend	React 18 + Vite + TypeScript	SPA, react-router v6; dev proxy /api → :8080
UI	Tailwind CSS + Radix UI (shadcn-style) + Zustand + TanStack Query	Server-state cached via React Query; one Zustand auth store
Realtime (client)	socket.io-client	Helpdesk tickets, kanban task board, timesheet events
API	Express 4 + TypeScript (Node 20)	All endpoints under /api; Zod validation; helmet, CORS, rate limit
ORM / DB	Prisma 5 · PostgreSQL 15	No repository layer — routes hold inline Prisma queries
Cache	Redis 7 (ioredis, optional)	OTP / password-reset rate-limit state; graceful skip if absent
Realtime (server)	Socket.IO 4	Shares the HTTP server with Express
Auth	JWT (httpOnly cookie, 7d) + bcryptjs	Email-OTP registration and password reset
Email	Nodemailer SMTP	Welcome, OTP, incident, deployment, reset, weekly report
PDF	PDFKit (in-process)	Weekly report generation
Integrations	Grafana API, Nginx Proxy Manager API	axios reverse-proxy (monitoring) and certificate sync (SSL)

1.2 Deployment topology

Figure 1 — Deployment topology (Docker Compose)



Solid arrows: in-process libraries / primary traffic. Dashed arrows: external HTTP calls or file storage. The frontend container only serves the built SPA; all API traffic is proxied by Nginx.

2 Module Inventory

Eight functional modules make up the product. The list below is the canonical one — it mirrors the frontend sidebar navigation, and each frontend route prefix maps 1:1 to a backend route file. Submodules are the feature areas inside each module.

2.1 Dashboard

Dashboard

Landing page after login; aggregates counts and recent items from almost every other module into one stats endpoint.

Submodule	What it does
Stats overview	Cross-module counters (projects, deployments, incidents, SSL, tickets…) from GET /api/dashboard/stats.
Recent activity	Latest activity-log entries and notifications surfaced on load.

Backend routes: dashboard.routes.ts · Main data models: read-only aggregates (User, Project, Incident, SslCertificate, …)

2.2 My Work Portal (Timesheet)

My Work Portal (Timesheet)

Personal work tracking: weekly timesheets with a rule-based approval chain, a calendar view, and an approval inbox.

Submodule	What it does
Timesheet	Weekly time entries per project/task; submit-week, recall, approve/reject flow.
Approvals	Approver inbox; approvers resolved by TimesheetApprovalRule (project_role / users / direct_manager / app_role) with self-approval escalation.
Calendar	Month view of entries and tasks (react-big-calendar).
Timesheet settings	Admin CRUD for approval rules.

Backend routes: timesheet.routes.ts (17 endpoints) · services/timesheetApproval.ts · Main data models: TimeEntry, TimesheetApprovalRule(+User), ProjectTask, Project

2.3 Projects & Assets

Projects & Assets

The delivery core: projects with kanban boards, team assignments, environments, credentials, documents, services and deployments.

Submodule	What it does
Projects	CRUD, members, finance tab; detail page with tabs (Kanban / Team / Finance).
Kanban tasks	Columns, drag-and-drop reorder, subtasks, comments — realtime via Socket.IO.
Environments	Per-project environment CRUD (nested under /projects/:id/environments).
Credentials	Encrypted credential store per project; /raw endpoint decrypts via crypto.
Documents	Project document storage with slugs + uploads.
Infrastructure nodes	Servers, databases, load balancers, k8s clusters.
Services & Applications	Service catalog tied to projects.
Deployments	Deployment records; creation fires notifications (+email).

Backend routes: project, credential, environment*, service, document, infrastructure, deployment, task, upload routes (*environment.routes.ts exists but is dead code — nested in project.routes.ts) · Main data models: Project, Environment,

TeamAssignment, ProjectTask(+Column/Comment/Subtask/Activity), Credential, Document, InfrastructureNode, Service, Application, Deployment

2.4 Network & Security

Network & Security

Internet-facing inventory and its safety: domains, SSL certificates (with live renewal through Nginx Proxy Manager), incidents and security findings.

Submodule	What it does
Domains	Domain inventory CRUD, synced from NPM hosts.
SSL certificates	Certificate inventory, expiry tracking, /renew action via NPM API.
NPM integration	/npm config/sync/test endpoints proxying an external Nginx Proxy Manager.
Incidents	Incident register; creation triggers notification + email fan-out.
Security findings	SecurityFinding records tracked alongside incidents.

Backend routes: domain, ssl, npm, incident routes · Main data models: Domain, SslCertificate, Incident, ChangeManagement, SecurityFinding

2.5 Finance & Procurement

Finance & Procurement

The largest module: a full procure-to-pay chain from budgets and requisitions through tendering, orders, receipts and payments to the asset register, driven by a shared approval engine.

Submodule	What it does
Budget control	BudgetItem / BudgetExpenditure tracking with per-budget summary.
Requisitions	Requisition forms + items; submit → multi-tier approval via approvalEngine (ApprovalPolicy tiers by division/amount).
RFQ / tendering	RequestForQuotation with items, send-to-vendors, quotation collection, quotation award.
Vendors	Vendor master data.
Purchase orders	PO + items lifecycle: submit → approve; reference numbers from generator.
Direct purchases	Skips tendering; same submit/approve flow.
Goods receipts	GR + items; inspection step.
Payments	Payment + attachments; submit/approve.
Assets	Asset register with transfer.
Internal projects	Non-billable internal work tracking.
Procurement settings	Admin: approval policies + tiers, reference-number configs, reference data, hidden Grafana dashboards.

Backend routes: budget, requisition, rfq, vendor, purchaseOrder, directPurchase, goodsReceipt, payment, asset, internalProject, settings routes · lib/numberGenerator.ts · services/approvalEngine.ts · Main data models: BudgetItem(+Expenditure), RequisitionForm(+Item), Vendor, RequestForQuotation(+Item), Quotation(+Item), PurchaseOrder(+Item), DirectPurchase, GoodsReceipt(+Item), Payment(+Attachment), Asset, InternalProject, ApprovalPolicy(+Tier), ApprovalAssignment, RequisitionApproval, ReferenceNumberConfig, ReferenceData

2.6 Helpdesk

Helpdesk

Internal IT ticketing with realtime comment threads and SLA-style reporting.

Submodule	What it does
Tickets	CRUD, status transitions, assignment, satisfaction rating.
Live comments	Ticket comments pushed over Socket.IO.
Categories & admin	HelpdeskCategory management (admin page).
Reports	Stats + monthly report endpoints backing the reports page.

Backend routes: helpdesk.routes.ts (17 endpoints, emits Socket.IO events) · Main data models: HelpdeskCategory, HelpdeskTicket, HelpdeskComment, HelpdeskAttachment

2.7 Operations & Monitoring

Operations & Monitoring

Observability: embedded Grafana dashboards and a full audit trail of every important action.

Submodule	What it does
Grafana	Config + dashboard/folder listing through the Grafana API; /proxy/* reverse-proxy for embedding iframes.
Activity log	Read API over ActivityLog; written by 26 route modules via logActivity middleware.

Backend routes: grafana, activity routes · services/pdf.service.ts (weekly report reads ops data) · Main data models: ActivityLog, Setting, Deployment, Incident (as report inputs)

2.8 Account & Administration

Account & Administration

Identity, access control and platform plumbing: auth with email OTP, Spatie-style RBAC, notifications, profile and uploads.

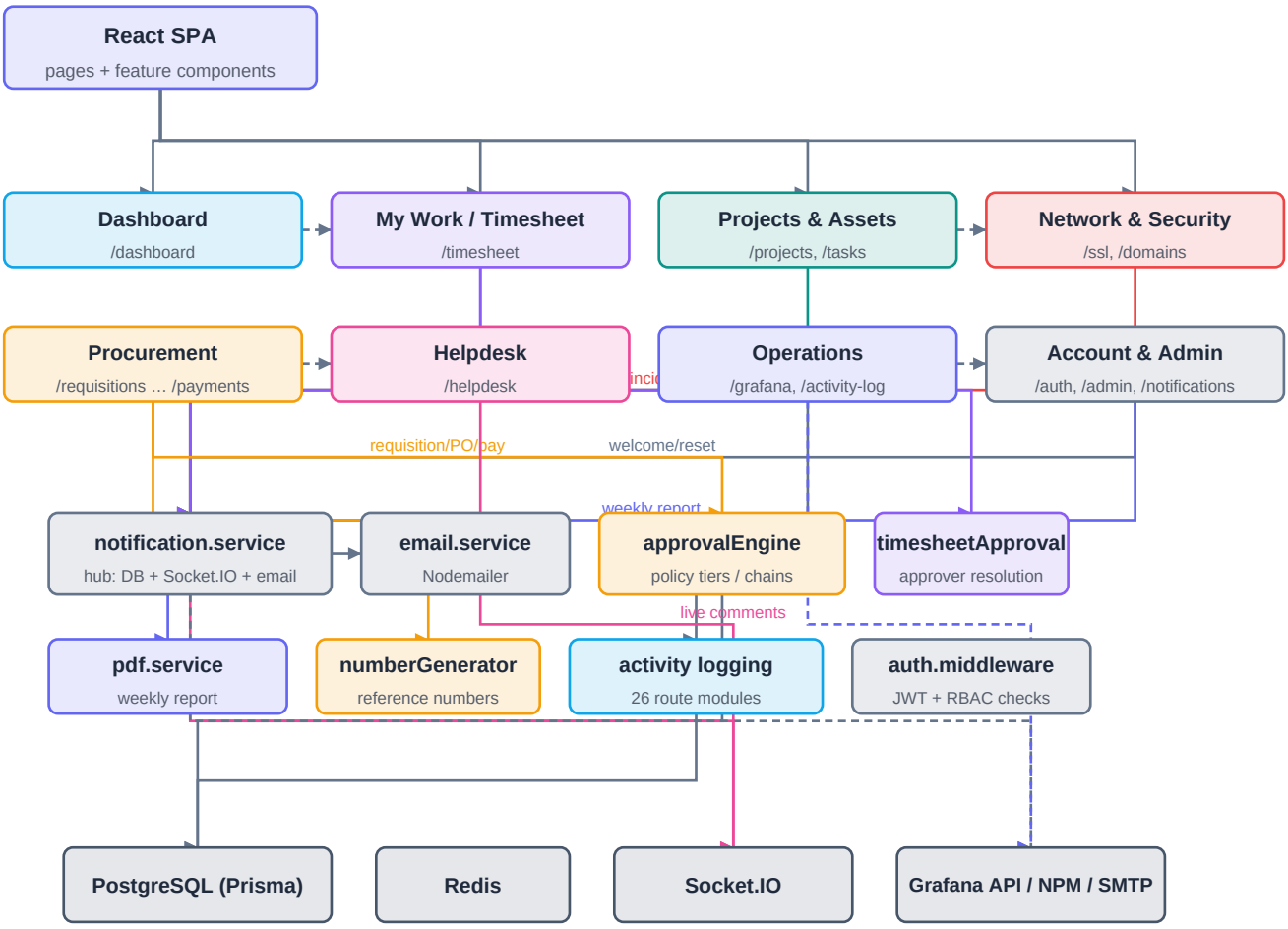
Submodule	What it does
Auth	Register (email OTP verify), login (JWT cookie), logout, forgot/reset password.
Users & roles (admin)	User CRUD + role assignment; Role/Permission tables (Spatie-style).
Notifications	Notification center: list, unread count, mark read, per-channel preferences; weekly-report PDF generation & test-send.
Profile	Own profile + password change.
Uploads	multer-based file upload/serve/delete under /upload.

Backend routes: auth, admin, notification, profile, upload routes · services/auth.service.ts, notification.service.ts, email.service.ts, rbac.service.ts (unused) · Main data models: User, PasswordResetToken, EmailVerification, Session, Role, Permission, ModelHasRole, ModelHasPermission, RoleHasPermission, Notification

3 Architecture — How Modules Connect

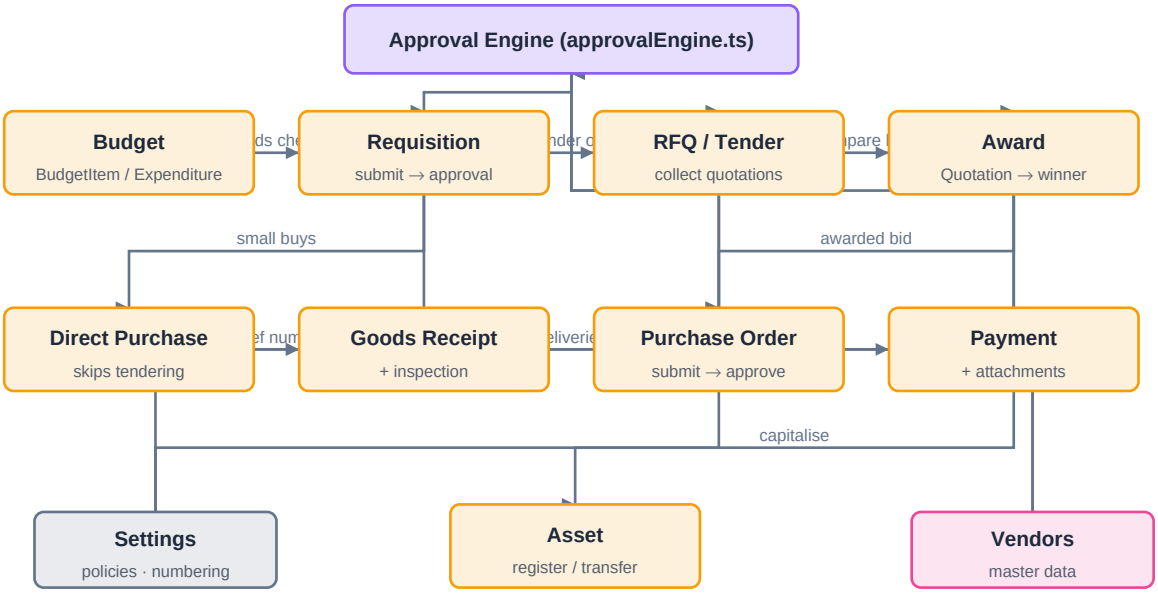
The backend is a modular monolith. Frontend pages never call each other; every arrow below is either a browser→API call on a route prefix, or an in-process dependency between backend route modules and shared services.

Figure 2 — Module connection map



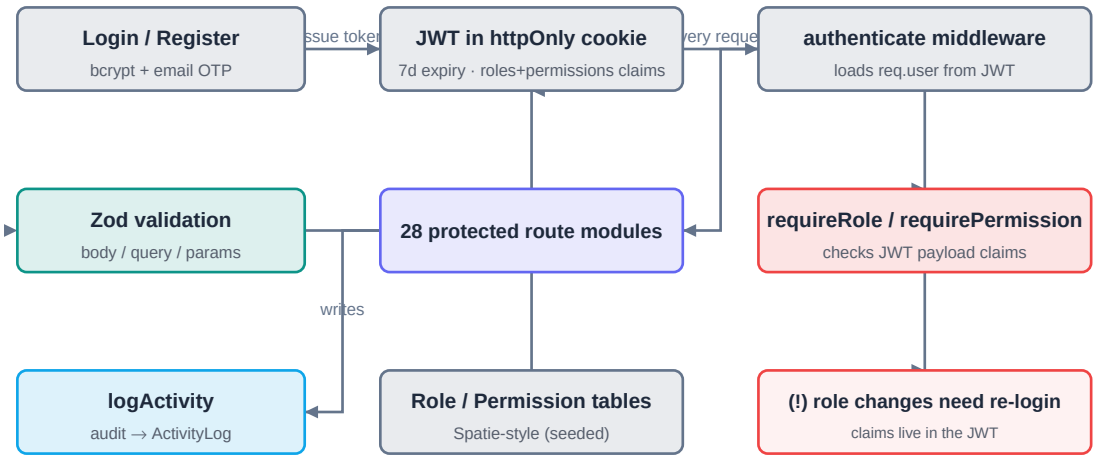
Dashed horizontal arrows: navigation siblings (no runtime coupling — coupling happens only through shared services and the database). Colored vertical arrows: the concrete cross-module dependencies found in the code (see §4).

Figure 3 — Procure-to-pay flow (Finance & Procurement module)



Requisitions, purchase orders, direct purchases and payments all run through the same approval engine, which matches ApprovalPolicy tiers (division + amount) and builds RequisitionApproval chains. Every document gets its reference number from ReferenceNumberConfig via lib/numberGenerator.ts.

Figure 4 — Authentication & RBAC flow



auth.service issues a JWT containing role/permission claims; middleware/rbac.ts authorizes against those claims rather than the database (services/rbac.service.ts exists but is unused). Authorization is enforced in 28 route files.

4 Shared Services & Middleware

Cross-module behavior lives in a small service layer. These are the only couplings between route modules — there is no route-to-route importing.

Service / middleware	Purpose	Consumed by
notification.service	Central notification hub: writes Notification rows, emits Socket.IO, composes emails	incident, deployment, admin, timesheet, notification routes; auth.service
email.service	Nodemailer transport + HTML templates (welcome, OTP, incident, deployment, reset, weekly report)	notification.service, auth.service
approvalEngine	Budget/purchase approval: evaluateApproval, createApprovalChain, processApprovalStep against ApprovalPolicy tiers	requisition route (PO/direct-purchase/payment flows reuse the policy tables)
timesheetApproval	resolveTimesheetApprover — rule-based approver resolution with self-approval escalation	timesheet route
pdf.service	gatherWeeklyReportData (cross-module Prisma aggregation) + PDFKit render	notification route (weekly report endpoints)
auth.service	register/OTP/login/getMe; orchestrates notification + email	auth route
activity (middleware/activity.ts)	logActivity — audit trail writer	26 route modules
rbac middleware	requireRole / requirePermission against JWT claims	28 route modules
validate middleware	Zod schema validation (body/query/params)	all routes
rbac.service	Spatie-style permission queries	**unused (dead code)** — enforcement is via JWT claims
lib/numberGenerator	Sequential reference numbers per ReferenceNumberConfig	6 procurement route modules
lib/socket	Socket.IO init + emitter	helpdesk, task, timesheet routes; task-activity lib

5 Data Model Overview (Prisma / PostgreSQL)

About 74 models in backend/prisma/schema.prisma, grouped by module:

Group	Models
Org / identity	Department, Position, EmploymentType, ProjectType, ProjectRole, User, PasswordResetToken, EmailVerification, Session
RBAC (Spatie-style)	Role, Permission, ModelHasPermission, ModelHasRole, RoleHasPermission
Platform plumbing	Notification, ActivityLog, Cache, CacheLock, Job, JobBatch, FailedJob, Setting
Projects / work	Project, Environment, TeamAssignment, TaskColumn, ProjectTask, TaskComment, TaskActivity, TaskSubtask, InternalProject
Timesheet	TimeEntry, TimesheetApprovalRule, TimesheetApprovalRuleUser
Infrastructure / ops	InfrastructureNode, Service, Application, Domain, SslCertificate, Deployment, Incident, ChangeManagement, BackupJob, DisasterRecoveryPlan, SecurityFinding
Procurement	BudgetItem, BudgetExpenditure, RequisitionForm, RequisitionItem, Vendor, RequestForQuotation, RfqItem, Quotation, QuotationItem, PurchaseOrder, PurchaseOrderItem, DirectPurchase, GoodsReceipt, GoodsReceiptItem, Asset, Payment, PaymentAttachment
Approval workflow	ApprovalPolicy, ApprovalPolicyTier, ApprovalAssignment, RequisitionApproval
Helpdesk	HelpdeskCategory, HelpdeskTicket, HelpdeskComment, HelpdeskAttachment
Misc	Document, Credential, ReferenceData, ReferenceNumberConfig

6 Noteworthy Observations

#	Observation	Detail
1	**Laravel heritage**	backend/vendor/ is a PHP Composer tree (Carbon, Laravel-style packages) left over from an earlier PHP version of this app — completely unused by the Node backend. The Prisma schema still mirrors Laravel/Spatie conventions (ModelHasRole, Cache, Job, FailedJob tables).
2	**Dead code**	environment.routes.ts is never mounted (environment endpoints are nested in project.routes.ts); services/rbac.service.ts is never imported (authorization uses JWT payload claims).
3	**No repository layer**	Route files hold inline Prisma queries; there is no data-access abstraction. Fine at this scale, but it means business logic and queries are interleaved.
4	**RBAC via JWT claims**	Roles/permissions are baked into the JWT at login; changing a user's role takes effect only at next login. The Role/Permission tables serve as claim sources and for admin management.
5	**Redis is optional**	Only auth rate-limiting (OTP/reset) touches Redis; the app degrades gracefully without it.
6	**Realtime scope**	Socket.IO is used only by helpdesk ticket comments, the kanban task board and timesheet events, over the same HTTP server as Express.
7	**Weekly report is in-process**	PDFKit renders the weekly report in the API process, aggregating incidents, deployments, projects, tasks and timesheets cross-module.
8	**Grafana embedding**	Monitoring is embedded Grafana iframes through an authenticated /grafana/proxy reverse-proxy rather than a native charting library.

Module map one-liner: **Dashboard** reads everything; **Projects & Assets** is the delivery core; **Network & Security** watches the edge; **Finance & Procurement** is the largest and most internally-connected module (a full procure-to-pay chain over a shared approval engine); **Helpdesk** and **My Work** are people-facing workflows with realtime and rule-based approvals; **Operations** observes and audits; **Account & Admin** provides identity and platform plumbing that every other module leans on.